

DAPPaaS: A Customizable Blockchain as a Service Platform for Decentralized Web3 Applications

Yubin Wang, Jiannong Cao, Shaocheng Chu, Hongbo Liu, Milos Stojmenovic, Shan Jiang

PII: S2096-7209(25)00150-2
DOI: <https://doi.org/10.1016/j.bcra.2025.100423>
Reference: BCRA 100423



To appear in: *Blockchain: Research and Applications*

Received date: 28 September 2025
Revised date: 17 October 2025
Accepted date: 25 November 2025

Please cite this article as: Yubin Wang, Jiannong Cao, Shaocheng Chu, Hongbo Liu, Milos Stojmenovic, Shan Jiang, DAPPaaS: A Customizable Blockchain as a Service Platform for Decentralized Web3 Applications, *Blockchain: Research and Applications* (2025), doi: <https://doi.org/10.1016/j.bcra.2025.100423>

This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: <https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article>. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2025 Published by Elsevier Ltd on behalf of Zhejiang University Press.
This is an open access article under the CC BY-NC-ND license
(<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

DAPPaaS: A Customizable Blockchain as a Service Platform for Decentralized Web3 Applications

Yuqin Wang^{a,b}, Jiannong Cao^b, Shaocheng Chu^b, Hongbo Liu^b, Milos Stojmenovic^c, Shan Jiang^{a,*}

^a*School of Software Engineering, Sun Yat-sen University, Zhuhai, China*

^b*Department of Computing, The Hong Kong Polytechnic University, Hong Kong SAR, China*

^c*Department of Computer Science and Electrical Engineering, Singidunum University, Belgrade, Serbia*

Abstract

Web3, referring to the next-generation decentralized web, has gained extensive attention from industry and academia. Its popularity drives considerable demand for continuous development and delivery of web3 applications and services. Blockchain is an essential technology that enables web3 but comes with considerable operational cost. Blockchain-as-a-service (BaaS) is considered as a promising solution for supporting web3 applications; however, existing BaaS platforms are conceptual, built for specific applications, or not optimized for emerging web3 applications. This work analyzes the primary features and fundamental requirements of decentralized web3 applications and articulates the critical role of blockchain for web3. To fill the gap, we introduce DAPPaaS, the first customizable BaaS platform specializing in web3 applications, with carefully designed goals and principles to meet the goals. Our key technical contributions lie on the approaches enabling component modularization, communication efficiency, automated scaling and performance monitoring, distributed deployment, and resource optimization. We implement a real-world application based on DAPPaaS and conduct extensive performance evaluations to validate its utility and efficiency.

Keywords: Blockchain, Web 3.0, Web3 applications, Web3 services, Blockchain as a service

1. Introduction

Blockchain is a distributed ledger technology for trustless, auditable data management [1]. In 2008, Nakamoto developed Bitcoin [2], a first of its kind of cryptocurrency and the first and most successful blockchain application. Later, academics and industries developed a wide range of applications based on blockchain, leveraging its distinctive features of decentralization, transparency, and immutability. Examples of application domains include cryptocurrencies [3], intelligent transportation [4], education [5], supply chain management [6], and health-care [7].

The development of blockchain technology mainly experienced three stages: 1.0, 2.0, and 3.0. Blockchain 1.0 is designed for recording transactions of Cryptocurrency, e.g., Bitcoin [8], related to money transfer, remittance, and digital payment. With the integration of smart contracts in blockchain systems, blockchain technology enters 2.0 era, in which the representative product is Ethereum [3]. Since 2.0, blockchain has received unprecedented attention world-widely, and enterprises have begun to develop numerous blockchain-based applications. While the blockchain technology attracting more attention from industry, enterprise-level customizable blockchain

solutions (blockchain 3.0) began to appear, in which Hyperledger Fabric [9] is the representative project.

More recently, decentralized applications built upon web3 has been identified as the killer application of blockchain technology [10]. web3 refers to the next-generation decentralized web where users have full control to their data and identity [11, 12]. Blockchain naturally meets the requirements of web3 and becomes an essential technology supporting decentralized web3 applications, e.g., DeFi [13], SocialFi [14], DAO [15], and DePIN [16]. According to CoinMarketCap, the market capitalization of DeFi was more than 118 billion USD in May 2024.

With the popularity of blockchain and web3 technology, there are increasing demands for developing and delivering blockchain-based web3 applications and services agilely and continuously. Moreover, many start-ups are eager to promote web3-related entrepreneurship even without enough budget for dedicated hardware. The technology push and the market pull give birth to the concept of blockchain as a service for decentralized web3 applications. It refers to a vendor's cloud-based blockchain and web3 infrastructure, allowing users to develop, host, and use their web3 components, functions, and applications in ease.

Many enterprises have launched BaaS platforms, among which Bitcoin, Ethereum, and Hyperledger Fabric are the most popular [17]. However, they are either limited in scalability or challenging to configure and customize [18]. In academia, researchers have proposed several BaaS solutions; however, they are conceptual [19, 20], designed for specific applica-

*Corresponding author

Email addresses: yuqin.wang@connect.polyu.hk (Yuqin Wang),
jiannong.cao@polyu.edu.hk (Jiannong Cao),
shaocheng.chu@connect.polyu.hk (Shaocheng Chu),
hong2liu@polyu.edu.hk (Hongbo Liu),
mstojmenovic@singidunum.ac.rs (Milos Stojmenovic),
jiangsh73@mail.sysu.edu.cn (Shan Jiang)

tions [21, 22, 23, 24, 25, 26, 27], or not optimized for real-world emerging web3 applications [28, 29]. The evolution of blockchain and web3 demands a customizable BaaS platform for decentralized web3 applications that existing platforms struggle to provide.

In this work, we propose DAPPaaS, the first customizable BaaS platform for decentralized web3 applications. The main contributions of this work are listed as follows:

- We present an in-depth analysis of features and requirements for decentralized web3 applications. Based on the analysis, we articulate why blockchain is an essential technology for such applications and design a generic system architecture for blockchain-based web3 applications.
- We design DAPPaaS, the first customizable BaaS system, focusing on building decentralized web3 applications agilely. System design includes design goals and principles, overall architecture, and detailed components. The key technical novelty lies in decoupling and modularization, automated scaling, performance monitoring, distributed deployment, and resource optimization.
- We implement DAPPaaS system, including network communication, data persistence, plug-and-play consensus, container-based deployment, and service monitoring tools. Furthermore, we showcase an example web3 application deployed on DAPPaaS and extensively evaluate system performance.

The remainder of this paper is organized as follows. Sec. 2 summarizes the related studies and projects and articulates the motivations of DAPPaaS. Sec. 3.1.3 introduces the design goals and principles of DAPPaaS to meet the requirements of decentralized web3 applications. Sec. 4 presents the design and implementation details. Sec. 5 evaluates the performance of DAPPaaS based on an example web3 application. Finally, Sec. 6 concludes this work and identifies the future directions.

2. Related Work

This section critically examines the prevalent blockchain as a service platforms for decentralized blockchain and web3 applications, delineating their adoption and limitations within both industrial and academic contexts.

2.1. Decentralized Web3 Applications

Recently, web3 has been increasingly popular worldwide. We sort out the essential web3 components and applications in Fig. 1 and categorize them into decentralized physical infrastructure networks (DePINs), decentralized autonomous organizations (DAOs), decentralized finance (DeFi), decentralized applications (DApps), and decentralized identity (DID).

DePIN lies in the bottom layer, connecting the web3 ecosystem with the physical world. DePIN tokenizes physical assets and decentralizes physical resources, making the assets and resources tradable and sharable in the web3 ecosystem. For example, real-world assets (RWAs), a popular web3 application in

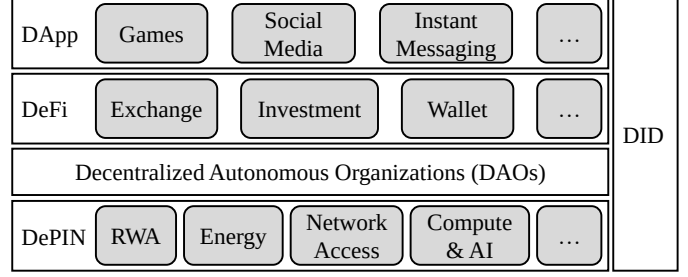


Figure 1: Layered web3 infrastructure with components and applications.

2024, make physical assets more accessible to a wide range of investors and democratize the investment and ownership. Traditional physical resources, e.g., electricity, network access, and computing resources, are usually supplied by centralized authorities. DePIN enables and incentivizes normal users to share spare physical resources with others. For instance, Helium is the first DePIN project that democratizes low-power and long-range network access of mobile devices, especially internet of things devices [30].

DAOs are the most popular web3 governance mechanism. Web3 applications are decentralized in nature regarding not only the deployment and running but the governance as well. Traditional centralized and even some blockchain-based applications are managed or governed by a single authority or a development community. In contrast, web3 democratizes the governance leveraging DAOs. Regarding a web3 application, the developers and users can vote for governance rules and automatically deploy them using smart contracts in DAOs. For instance, Aragon is a framework to establish new DAOs and tailor them to specific needs; MakerDAO is a DeFi-related DAO responsible for the maintenance and regulation of Dai, a famous stable coin [31].

DeFi removes centralized authorities from financial transactions and empower tokens and cryptocurrencies with functions similar to fiat currencies. DeFi is critical for web3 because everything is tokenized in web3 and the healthy circulation of tokens demands DeFi. Popular DeFi applications include exchanges to trade cryptocurrencies, wallets to manage digital assets, investment tools for savings, loans, and venture capitals [32]. DApps build at the top with the support of the underlying layers. There are a wide range of web3 DApps, including games, social media, and instant messaging.

DID is another important web3 component vertical to DApp, DeFi, DAO, and DePIN. DID refers to unique identities of web3 users. DID is vital to achieve the ultimate goal of web3, i.e., to make users capable of fully controlling their identities and data. In the physical world, people have identity cards and passports issued by governments, making each individual identifiable. However, user identification becomes challenging in web3 because no central authority (e.g., governments) should be relied on and digital identities can be easily faked. Secure implementation of DID is still an open issue [33].

2.2. Blockchain as a Service Platforms

Within the industry, Bitcoin [34], Ethereum [3], and Hyperledger Fabric [9] serve as archetypal platforms representing the evolutionary stages of blockchain technology, namely Blockchain 1.0, 2.0, and 3.0+ respectively.

Bitcoin, conceived in 2008, stands as the inaugural application of blockchain technology, whose market capitalization exceeded 1 trillion US dollars in 2021. Despite its monumental success, Bitcoin is criticized by its scalability issues, lacking support for smart contracts, and challenges in customization. Consequently, enterprises seldom select it as a underlying infrastructure for deploying decentralized web3 applications.

Ethereum, launched in 2014, introduced smart contract functionality, enabling developers to create decentralized applications at ease. In November 2015, Microsoft unveiled Ethereum blockchain as a service (EBaaS) on its Azure cloud platform. EBaaS provides a cost-effective, plug-to-use environment for financial service developers to accelerate the development and testing for their blockchain applications. It facilitates creating private, public, or consortium blockchain networks while leveraging Azure's globally distributed infrastructure to deploy blockchain products. Despite the advancements on EBaaS, Ethereum has certain deficiencies, including limited scalability and high implementation difficulties.

Hyperledger Fabric represents a permissioned blockchain framework with a modular architecture that allows for distinct nodes' roles, execution environments for smart contracts, and configurable consensus and membership services. IBM's blockchain network, for instance, is constructed atop Hyperledger Fabric framework. Although rapid development is conducted upon Hyperledger Fabric community, it predominantly focusing on support of private blockchain platforms, offering a limited selections on consensus mechanisms. What's more, it also poses a steep learning curve for developers with limited blockchain experience.

In addition to BaaS platforms developed for Bitcoin, Ethereum, and Hyperledger Fabric, major corporations such as Amazon, Baidu, and Alibaba have ventured into developing their own BaaS platforms, integrating these offerings into their broader cloud service portfolios. These enterprise-level BaaS solutions aim to streamline the adoption of blockchain technology across various industry sectors.

In academia, researchers have also been developing BaaS solutions. In [19], Samaniego et al. propose blockchain as a service for the first time. However, it is conceptual and only shows a set of experimental results. Singh et al. analyze the management, governance, and trust issues of a BaaS platform while merely touching on the technical details [20]. In [21] and [22], Alia et al. and Aujla et al. consider the integration of BaaS with end-edge-cloud networks and software-defined networks for applications in drones and smart cities, respectively, which are pioneering but lack design details of the BaaS.

Recent scholarly contributions have introduced two innovative BaaS platforms, namely NutBaaS [29] and uBaaS [28]. NutBaaS offers a comprehensive suite of blockchain services within cloud computing environments, encompassing network

deployment, system monitoring, and the analysis and testing of smart contracts. These provisions enable developers to concentrate on crafting business-specific code, thereby facilitating the tailored application of blockchain technology to distinct business contexts without the encumbrance of system maintenance and monitoring. Despite NutBaaS's well-defined system architecture and its technical strategies aimed at bolstering reliability and security, the delineation of its various architectural layers remains ambiguous.

As for uBaaS, it presents a diverse array of services, including deployment as a service, design pattern as a service, and various auxiliary services. uBaaS's deployment mechanism is designed to operate independently from specific cloud service providers or blockchain platforms. The efficacy and scalability of uBaaS have been substantiated through a real-world case study focused on quality tracing. uBaaS distinguishes itself through a vendor-neutral approach and furnishes detailed insights into its implementation. Nonetheless, uBaaS offers limited performance in terms of security, flexibility, and other critical metrics when compared with existing solutions.

While uBaaS and NutBaaS pioneered modular, microservice-based, and cloud-agnostic blockchain services, their design was primarily aimed at general enterprise adoption. In contrast, DAPPaaS proposed in this work is fundamentally web3-native. Its design principles, from component modularization to resource optimization, are explicitly derived from an analysis of web3-specific requirements.

Similar studies include blockchain as a service for cloud-edge collaboration [25], local 5G operators [24], supply chain optimization [26], internet of things [27], software-defined networks [22], and business collaboration [23]. All these BaaS solutions are conceptual, for specific applications, or not optimized for emerging web3 applications. It is important to design and develop a generic and customizable blockchain as a service platform specializing in decentralized web3 applications.

3. DAPPaaS System Design

This section analyzes the requirements of web3 applications and introduces the design goals and principles to meet the requirements.

3.1. Requirements of Web3 Applications

In this section, we are going to briefly introduce the evolution of web technologies, articulating the necessity and general architecture of employing blockchain technology for web3 applications, and finally identifies the requirements of blockchain-based web3 applications.

3.1.1. Evolution of Web Technologies

As depicted in Fig. 2, web technologies have gone through three generations from read-only static web1 and read-write interactive web2 to read-write-own intelligent web3.

The inaugural iteration of the world wide web, known as web1, was conceived by Tim Berners-Lee in 1989 [35]. It

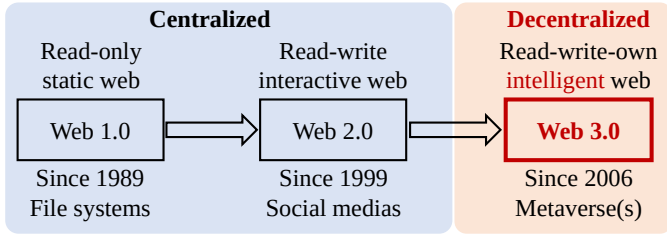


Figure 2: The evolution of web technologies and their representative applications: file systems in web1, social media in web2, and metaverses in web3.

was designed as a global hypertext space with a unique identifier for each accessible document. It established the prototype for a “read-only” web characterized by static pages on internet service providers or free hosting services with minimal user interaction. In the web1 era, information flow on the internet was unidirectional: akin to traditional media, where businesses and editors curated content for passive consumption. Content ownership is relegated to the web page proprietors. Despite its limitations, web1 was a pivotal development, centralizing information access and setting the stage for the subsequent evolution into the interactive web2.

Web2, coined by Tim O’Reilly, signifies a paradigm shift in web utilization rather than a mere technological upgrade. Web2 transitioned from a static “read-only” platform to an interactive “read-write” medium, enabling user-generated content and participatory information exchange [36]. This evolution introduced bidirectional information flow, with users contributing to and shaping online content. Web2 has become synonymous with user-centric platforms that foster creation and sharing, exemplified by social networks and content platforms. This shift has catalyzed an unprecedented surge in user-generated content, facilitating new forms of connection and socialization. Web2 has thus been instrumental in transforming the internet into a dynamic conduit with significant societal implications for communication, collaboration, and creativity.

In 2006, Tim Berners-Lee introduced the concept of web3 as the semantic web, aiming to make data on the web machine-readable, thereby enhancing the web’s intelligence [37]. Web3 envisions a structured data environment where information can be easily discovered and integrated across applications, stored in semantic frameworks for efficient reuse [38]. Gavin Wood, a co-founder of Ethereum, proposed an alternative web3 vision in 2014, advocating for decentralized, user-owned web services rather than centralized corporate hosting. This iteration of web3 emphasizes user sovereignty over data and leverages blockchain technology to facilitate this shift.

Despite web2’s focus on social interaction and user-generated content, users typically do not retain ownership of their contributions, leading to a democratic deficit. The value of data is significant, with large enterprises often benefiting disproportionately from big data advantages and monetization opportunities while users unknowingly forfeit their data without compensation. Web3 seeks to redress these imbalances by ensuring users retain ownership and control over their digital content and data [39]. It challenges the current centralized data

storage paradigm, which is plagued by bandwidth bottlenecks, scalability issues, single points of failure, and privacy and security vulnerabilities. The decentralized, user-centric data storage model of web3 addresses these concerns, reducing the risk of single-point-failure and service disruptions while enhancing the privacy. In summary, web3 heralds a more intelligent, democratic Internet, empowering users with data control and ownership and mitigating centralization-related risks.

3.1.2. Blockchain for Web3 Applications

In web3, the user’s data is owned by the user, and the user can decide where and how the data is stored. User data can be reused on different web pages. No company can store users’ data centrally. Therefore, web3 must adopt a distributed storage strategy [40]. While both blockchain and distributed databases are candidates for the requirement, blockchain would further ensure security and reliability compared to the latter one. Nevertheless, blockchains are more suitable for applications in a trustless environment, and distributed databases are more suitable in performance critical scenarios [41, 42, 43]. Since the stored content may contain sensitive user information, it is hard to build trustworthy relationship among companies and users. For security and privacy concerns, blockchain is more suitable for web3 than a distributed database.

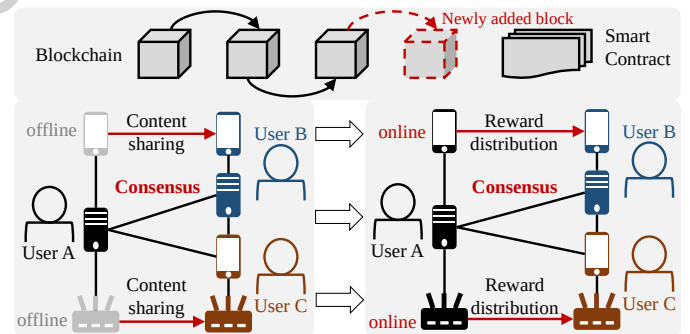


Figure 3: Architecture of blockchain-based web3. Users share content via users’ devices instead of centralized servers. Blockchain and smart contracts incentivize content sharing and automate reward distribution.

To this end, we propose blockchain-empowered web3, which preserves Berners-Lee’s concept that web pages should be machine-readable [37]. On this basis, Blockchain-empowered web3 also requires the separation of applications and user data and the use of blockchain to decentralize data storage so that users can control their data. Fig. 3 depicts the system architecture of blockchain-empowered web3. Inside, users enjoy web services similarly as in web2; however, content sharing in web3 is normally achieved through P2P communication among users’ devices rather than pulling from centralized servers, e.g., the cloud. Blockchain technology establishes trust among users and devices that do not trust each other, where consensus mechanisms and smart contracts would help to incentivize content sharing and automate reward distribution process.

3.1.3. Requirements of Blockchain-based Web3 Applications

We identify and summarize three primary requirements of blockchain-based web3 applications as follows:

- *Platformless.* This term refers to the architectural paradigm of web3, which is defined by the absence of a centralized intermediary or a single controlling entity. The focus is on designing systems that remove single points of control and opportunities for rent-seeking from the application's core logic and data management. In essence, "platformless" describes the non-intermediated nature of Web3 applications, where interactions occur directly between peers according to protocol rules rather than being arbitrated by a central authority.
- *Autonomous.* As a direct consequence of a platformless architecture, "autonomous" describes the empowerment and rights granted to the end-user. It refers to the user's sovereign control over their own digital identity and data. While "platformless" defines the system's technical structure, "autonomous" defines the user's experience within that structure. Users are no longer subjects of a platform but are empowered participants who own and manage their digital footprint.
- *Tokenized.* Tokens can represent any kind of data with value, including cryptocurrencies, real estate, intellectual property, and even in-game assets. The web3 Tokenization Principle emphasizes that assets and services are tokenized, allowing automatic payment as they are used, which promotes interoperability by incentivizing standardization. In essence, tokenization in web3 provides a mechanism for ownership, governance, and efficient exchange within decentralized networks, extending beyond traditional domains like domain names to a wide range of assets and services.

3.2. Design Goals of DAPPaaS

This section presents the four design goals of DAPPaaS that echo the requirements of web3 applications identified in Sec. 3.1.3.

3.2.1. High Customizability

DAPPaaS, as a BaaS platform for decentralized web3 applications, regards high customizability as core feature to satisfy various demands for different use cases. With high customizability, users from various industries can choose appropriate components to build their own projects. In addition, the consensus algorithm significantly impacts the system throughput and security, so the system should support to assemble multiple consensus algorithms.

3.2.2. High Availability

Successful cloud computing platforms usually have high availability because enterprise-scale applications must be consistent and stable. Take the financial service as an example: a few hours of system downtime may result in millions of dollars

in loss. In order to have high availability, the system should be fault-tolerant and eliminating the possibility of having a single point of failure. The single point of failure is referred to as a design flaw in which errors of a single component will shut the entire system down. High availability empowers the system to resist various external attacks, while ensuring the system to run continuously and stably even with internal faults.

3.2.3. High Performance

Although Bitcoin and Ethereum are arguably the most well-known blockchain systems, they have not been used as the primary method for daily payment. One major factor that hinders Bitcoin and Ethereum from being a daily payment method is their unacceptable delay on finalizing transactions. Particularly, confirming transaction on Layer 1 (base blockchain network infrastructure) with strongest security insurance, for Bitcoin and Ethereum, the blockchain system can only process around seven and twenty transactions per second, respectively. Such a system throughput needs to catch up to the speed of 24,000 transactions per second required by modern financial systems. Besides high system throughput, the system should remain high-performance even when the workload increases [44]. It is embodied in the reasonable arrangement of resources and optimization on hardware allocation.

3.2.4. Effortless Deployment

Software deployment can be cumbersome, especially for a distributed system. Large cloud computing corporations have dedicated teams responsible for system deployment and maintenance, which may differ from small corporations due to economic concerns. DAPPaaS aims to provide an effortless deployment process to deliver blockchain technology to a broader group so that a few straightforward commands can initiate the blockchain system with web3 services and applications. Such design goal corresponds with agile development and continuous delivery principles in software engineering.

3.3. Design Principles of DAPPaaS

This section introduces the main design principles and summarizes the technical novelty of the DAPPaaS system. Tab. 1 summarizes how the design goals are fulfilled by the design principles.

Table 1: The design principles of DAPPaaS to fulfill the design goals.

Design goal	Corresponding design principles
High customizability	Component modularization
High availability	Component modularization Distributed deployment
High performance	Efficient network communication Resource optimization
Effortless deployment	Automated scaling and performance monitoring Distributed deployment

3.3.1. Component Modularization

The first principle is modularizing components. Each component is independent, so users can customize each component individually. Meanwhile, such a design principle carries significant advantages:

- *Loose coupling.* Coupling refers to the degree to which components of an application rely on each other. When strong coupling occurs, modifications on a single component will also influence other components; this brings unnecessary work and unwanted modifications to other components. Whereas a loosely coupled application only has weakly associated components, and changes to one component do not affect the others.
- *Agile development.* In the industry, the efficient teamwork is crucial to success. With modularized components, a team can split the work easily; meanwhile, team members can work on different components in parallel.
- *Easy maintenance.* A single component can be updated and tested individually with self-contained components. Besides, less complicated parts are less error-prone and easier to debug.

DAPPaaS offers four customizable component categories: storage, network, consensus, and application. As discussed, high customizability is one of the design goals of DAPPaaS, and we argue that customization can be achieved by developing new components using standard application interfaces (APIs) and integrating them with existing components to compose web3 applications.

3.3.2. Efficient Network Communication

To improve the overall performance of a distributed system, we need to minimize network communication costs. Network latency generally refers to the delay in carrying messages from one place to another. Generally speaking, a low-latency network can contribute to higher system throughput. The network is powered in a peer-to-peer (P2P) fashion for blockchain-based applications, where every node simultaneously acts as a client and a server. As for efficient communication among peers, DAPPaaS uses the remote procedure call (RPC) [45] to implement network communication. RPC is a protocol that allows a node to call another node's functions without knowing if the actual function works in a remote manner. The process of calling a remote function should be transparent to the user. RPC uses the transmission control protocol (TCP) as the underlying transport protocol. Besides, the hypertext transfer protocol (HTTP) is also built on top of TCP. HTTP is usually packed with excessive information, which results in a slower transmission speed than RPC. Therefore, RPC is more suitable for scenarios that require higher network performance.

3.3.3. Automated Scaling and Performance Monitoring

Traditional manual software deployment requires engineers to be involved in the entire process, which is inefficient; besides, the deployment processes are cumbersome and vulnera-

ble to human mistakes. Instead, effortless deployment means all services are deployed and managed automatically.

Meanwhile, having automated service deployment also contributes to high availability. The single point of failure is the enemy of high availability, mitigating the problem involves deploying redundant instances. Once an instance goes down, other instances should take over as soon as possible. Having a human monitoring all instances and manually swap the malfunctioned instance is not ideal. Instead, automatic deployment should detect and resolve errors effortlessly.

Automated scaling may help allocate system resources on demand to further increase system performance. Automated scaling is a method that dynamically adjusts computational power depending on the current service load. Take Amazon as an example; the website reaches an exceptionally high load during the Black Friday sales. The service can be overloaded if the infrastructure cannot automatically increase the number of instances. Conversely, deploying many instances during the low tide period can result in unnecessary cost of computational resources.

3.3.4. Distributed Deployment

Deploying web3 applications presents a significant challenge in existing blockchain infrastructure, particularly within the Ethereum ecosystem. In Ethereum, the deployment of smart contracts necessitates that developers first compile their Solidity code into bytecode, subsequently utilize APIs for deployment, and ultimately employ the web3 library to invoke the contracts. This process can be notably cumbersome for individuals not well-versed in blockchain technology.

As a BaaS platform, DAPPaaS offers a variety of deployment methodologies that are accessible through web interfaces. This approach enables developers to convert their developed components into an executable program, an installation script, or a Docker image. They can upload and configure these components, forming a blockchain network. Furthermore, our platform facilitates the initialization or updating of components or nodes alongside their monitoring. This deployment strategy significantly enhances the platform's flexibility, making it more accessible and user-friendly for developers, regardless of their familiarity with blockchain technology.

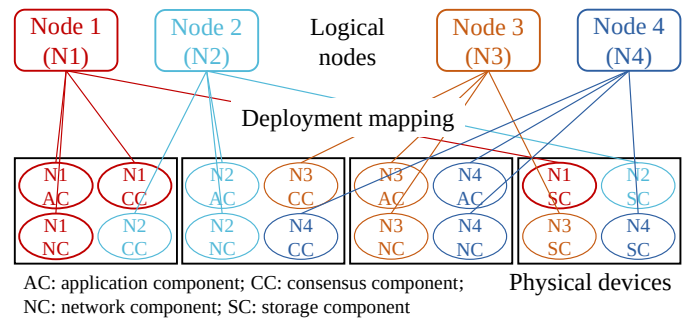


Figure 4: Deployment model of DAPPaaS. A logical node consisting of four components can be deployed on different physical devices.

In this work, we introduce an innovative distributed deploy-

ment scheme designed to enhance the reliability and security of the system. The scheme is designed on separation between logical nodes and physical machines. A logical node comprises four distinct components, which can be distributed across various physical machines for deployment. As illustrated in Fig. 4, the application, network, and consensus components of logical node 1 are situated on physical machine 1, whereas the storage component of the same logical node is located on physical machine 4.

This deployment strategy offers a significant advantage in its resilience against single-point attacks. The failure or compromise of a single component does not jeopardize the functionality of the entire blockchain network, thereby ensuring continuous operation. Furthermore, this scheme allows for the strategic placement of components based on the specialized capabilities of physical machines. For instance, storage components with superior storage capacities can be optimally deployed on physical machines. This tailored deployment not only leverages the inherent strengths of each physical machine but also contributes to the overall efficiency and robustness of the system.

3.3.5. Resource Optimization

In this work, we advocate for an optimization of component deployment subject to the computational power of physical nodes. To ensure the scalability of the platform, we employ two primary methodologies.

First, each principal component is conceptualized as a portable distributed subsystem. This design allows the components to be instantiated across different threads, processes, or host machines. For instance, application and consensus components may be parallelized to enhance transaction processing throughput. Conversely, components like the storage component may be distributed across nodes to accommodate extensive querying demands and manage large volumes of data. Communication between components is standardized through predefined protocols and frameworks, facilitating system scalability and cross-platform interoperability.

Secondly, we emphasize the re-usability of components within the platform. We supply a suite of predefined components, such as a proof-of-work consensus component, which developers can readily employ to establish their networks initially. Additionally, developers have the flexibility to implement their self-created components. This strategy not only diminishes development costs but also affords greater versatility in application design, thereby fostering an environment conducive to innovation and efficiency in blockchain network development.

4. DAPPaaS System Architecture and Component Design

This section applies the principles in Sec. 3.3 for designing the overall system architecture of DAPPaaS with loosely coupled components. Then, we showcase an example application built on DAPPaaS.

4.1. System Architecture

Fig. 5 illustrates the system architecture of the DAPPaaS. The system can be mainly divided into two parts: user applica-

tion and server instances cluster. User application provides the necessary functions and interfaces to interact with a user. As application data is stored in the blockchain, the user application needs to fetch application data from a node server instance.

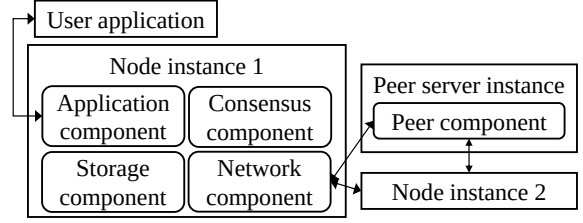


Figure 5: The system architecture of DAPPaaS, consisting of user applications, node instances (application, consensus, storage, and network components), and peer server instances (peer components).

The node instance serves blockchain and application logic. Once a user sends requests to a server node, the application component handles incoming requests and generates corresponding transactions. The consensus component adds transactions inside the transaction pool and mines a block to store transactions. During the block mining process, nodes need to share data and communicate with each other to reach a consensus. The network component provides node communication. After transaction confirmation, data is stored on the blockchain and persists via storage component. The peer server is the registrar for all node instances; a new node that joins the P2P network needs to register at the peer server. Besides, the peer server assists in reaching a consensus using algorithms such as practical byzantine fault tolerance (PBFT) and proof of work (PoW). During a leader election, the peer server collects all voting information and elects a leader based on pre-defined rules.

The lifecycle of a typical transaction begins when a user request originates from the user application. This request is first processed by the application component, which generates a corresponding transaction. The newly created transaction is then passed to the consensus component, where it is added to the transaction pool. To achieve network-wide agreement, the consensus component collaborates with the network component to broadcast the transaction and participate in the consensus process, which involves peer-to-peer communication with other nodes for tasks such as block proposals and voting. Once consensus is reached on a new block, the consensus component finalizes it and passes the confirmed data to the storage component for persistent storage. Finally, to complete the cycle and reflect the updated state to the user, the application component can query the storage component to retrieve the newly recorded data.

4.2. Component Design

There are two types of instances within a cluster: the node and the peer. All node instances in the P2P network communicate with their network components. In other words, one node can only call interfaces provided by another node's network component. Additionally, when a node joins and exits the network, its network component reaches the peer server.

The other three components in a node instance are not directly exposed to other nodes. The application component communicates with the consensus component and the storage component. When the application component receives a new request, it generates a new transaction and passes it to the consensus component. When the application component needs application data, it reaches the storage component for data. The consensus component communicates with the application component and the network component. Whenever there is information to share, the consensus component passes the information to the network component for broadcasting. Similarly, when the network component receives new information, the information is passed to the consensus component for verification.

4.2.1. Storage Component

The storage component is shown in Fig. 6, responsible for persisting web3 data in the database. It has two APIs: 1) “saveData” to save data into the database, and 2) “getData” to quarry data from the database. When the application component needs specific application data, it calls the “getData” interface of the storage component. Likewise, the newly generated data is saved by calling the “saveData” interface.

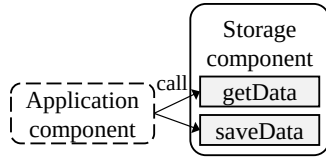


Figure 6: The storage component in DAPPaaS with two APIs callable by application components.

4.2.2. Peer Component

The peer component, as depicted in Fig. 7, handles the registration and exiting of a node in the P2P network. It has three APIs: 1) “joinNetwork” to join the network, 2) “exitNetwork” to exit the network, and 3) “getAllNodes” to get a list of all nodes within the network.

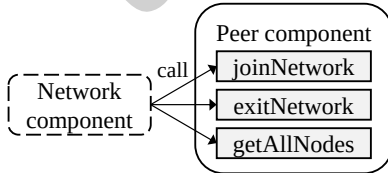


Figure 7: The peer component in DAPPaaS with three APIs callable by network components.

In addition, when using the PBFT consensus algorithm, the peer component has an optional interface: “startElect” to start a leader election. For a new round of PBFT, node instances call the “startElect” interface to elect a leader. The election component gathers all participants’ information and elects a leader based on pre-defined rules for the current round.

4.2.3. Network Component

Fig. 8 shows the network component of a DAPPaaS node, which is used for communication within the P2P network. It has nine APIs: 1) “broadcastTransaction” to broadcast a transaction to its peers, 2) “receiveTransaction” to receive a transaction from its peers, 3) “broadcastBlock” to broadcast a block to its peers, 4) “receiveBlock” to receive a block from its peers, 5) “updateLeader” to receive the information of the newly elected leader, 6) “broadcastPrepare” to broadcast a prepare to its peers, 7) “receivePrepare” to receive a prepare from its peers, 8) “broadcastCommit” to broadcast a commit to its peers, and 9) “receiveComit” to receive a commit from its peers.

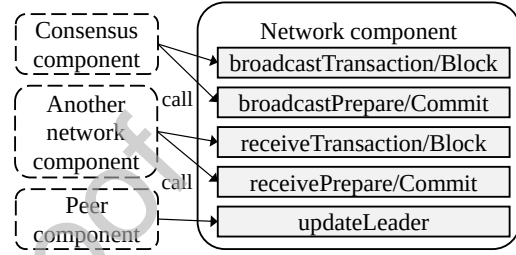


Figure 8: The network component in DAPPaaS with nine APIs callable by consensus, peer, and other network components.

When the consensus component receives a new transaction, the consensus component calls the “broadcastTransaction” interface of the network component, and the network component broadcasts the transaction to all nodes. The “receiveTransaction” interface is called by other nodes’ network components for receiving a transaction. Similarly, the “broadcastBlock” and “receiveBlock” interfaces are for sending and receiving blocks within the P2P network.

As the PBFT algorithm needs additional communication to reach a consensus, the rest of the interfaces are for PBFT only. All active nodes reach the peer server each round and register for the leader election. After the peer server chooses a leader, it updates the leader information by calling the “updateLeader” interface of the network components for all nodes. Then, the leader proposes a block and broadcasts the new block in the network. Other nodes receive the block and verify its authenticity. Like transmitting the information of transactions and blocks, the “prepare” and “commit” messages regarding PBFT are transmitted by calling “broadcastPrepare”, “broadcastCommit”, “receivePrepare”, and “receiveCommit” interfaces.

4.2.4. Consensus Component

The consensus component, as shown in Fig. 9, is responsible for reaching a consensus among nodes. It has four APIs: 1) “onNewTransaction” to handle a new transaction, 2) “onNewBlock” to handle a new block, 3) “packTransaction” to pack transactions inside the transaction pool into a block, and 4) “getTransactionPool” to get all transactions inside the transaction pool.

Once the application component generates a new transaction, it calls the “onNewTransaction” interface of the consensus component to add the new transaction to the transaction

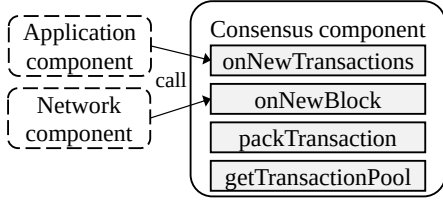


Figure 9: The consensus component in DAPPaaS with two APIs callable by application and network components and two internal overridable APIs.

pool. Then, the consensus component executes the “packTransaction” interface, which can be re-defined by users [46], to pack transactions within the transaction pool into a new block. When the current node receives a new block, the network component calls the “onNewBlock” interface of the consensus component to verify the block and update the blockchain. The “getTransactionPool” interface also checks all unconfirmed transactions within the transaction pool.

4.2.5. Application Component

Fig. 10 elaborates on the application component in DAPPaaS that handles user requests. It has three APIs: 1) “generateTransaction” to generate transactions based on user inputs, 2) “getAppData” to fetch application data to display, and 3) “generateAppData” to extract application data from transactions.

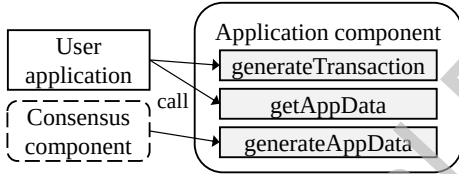


Figure 10: The application component in DAPPaaS with three APIs callable by user applications and consensus components.

When a user interacts with the application, the user interface displayed data is fetched by calling the “getAppData” interface of the application component. After taking user requests, the “generateTransaction” interface is called to generate corresponding transactions. Besides, after the consensus component verifies a newly received block, the consensus component calls the “generateAppData” interface of the application component to extract application data from transactions.

4.3. System Implementation and Example Application

Detailed implementation of DAPPaaS will be shown in this section, including network communication, data persistence, plug-and-play consensus, container-based deployment, and service monitoring tool with a decentralized web3 application.

4.3.1. Example Application: Transcript Management

A transcript management system is essential for the education institution to manage student transcripts. Many universities have several departments and offer various programs; at the same time, thousands of students and staffs are involved in daily teaching activities. Managing transcripts for all students

can be tedious. Currently, some institutions have already established their dedicated transcript management system. The system helps academic staff to keep track of transcripts efficiently and eliminates unnecessary errors without manual labor.

Although such system may work well internally, it barely helps students transferring their transcripts for personal development. For example, a typical credit transfer process involves applying for a transcript from the original school and sending the required documents to the target school. This process may be time-consuming, and the student may have to pay additional fees on document purchasing and shipping.

Another problem with the dedicated transcript system is the manipulated documents. Some students who fail to attain their degrees may purchase forged documents illegally. Even worse, verifying the authenticity of enormous documents may take unacceptable time and additional effort.

In this work, we propose PolyScript, a decentralized web3 application managing students’ transcripts. PolyScript aims to provide a generic platform to manage student transcripts. Academic staff and students from different institutions can join the same system. With the consent from a user, transcript information can be shared easily among institutions. Since all the application data is stored in a decentralized system, security and privacy could be ensured. As records stored in blockchain are immutable, forged transcript information is also prevented.

We implement PolyScript as a web-based application. Since PolyScript is built on top of DAPPaaS, it will have two types of servers: domain server and node server. Each node server contains four components: application component, consensus component, network component, and storage component. The application information is stored in a P2P network. The data stored in each node is synced across the network. The application component takes in a new student record and then packs the transcript into a transaction. The consensus component puts the transaction with transcript information into a transaction pool. After a certain time, the consensus component packs transactions in the pool into a block. Then, the network component shares transcripts and blocks information within the network. Once a transcript is finalized, the storage component persists in the record.

To validate real-world usability, we recruited 12 sophomore software-engineering students and asked each of them to deploy a personal transcript vault on PolyScript without prior blockchain experience. After a 15-minute guided walk-through, they completed a System Usability Scale (SUS) questionnaire and an open-ended debrief. The aggregate SUS score was 84.5/100 (excellent), with 11/12 participants agreeing that “the one-command Docker launch removed the usual Web3 setup fear.” Open feedback highlighted three points: (1) “The Kubernetes dashboard made it obvious when my node was syncing, i.e., no black-box feeling,” (2) “I could swap PBFT for PoW with a single drop-down, i.e., pretty cool for class experiments,” and (3) two students requested a GUI drag-and-drop for smart-contract compilation instead of the current CLI. All students successfully transferred a mock transcript to a peer node within three seconds and verified its hash on the explorer; none reported confusion about gas or private-key management due

to the built-in custodial wallet feature. These responses confirm that PolyScript, as instantiated on DAPPaaS, lowers the entry barrier for novice developers while still exposing enough configurability to be pedagogically valuable.

4.3.2. Node Server

The back-end server running in each node is responsible for handling incoming requests and blockchain services. It contains multiple node instances and a peer server instance. Meanwhile, each instance includes several components.

In order to adopt the modularized component design principle, PolyScript is developed using a microservice architecture, with each component being a dedicated service. Microservices means the project consists of multiple independent services communicating through well-defined APIs, each responsible for a distinct task [47]. The microservice architecture allows the application to be deployed, updated, and scaled for each service to meet the demands of specific functionality.

4.3.3. Network Communication

In a distributed system, nodes work together to accomplish specific tasks. Different nodes may be hosted on different servers, so a network is required to communicate between nodes. The network communication cost should be minimized to improve the overall system performance. The traditional way of transferring data is using HTTP protocol. HTTP is an application-layer protocol based on TCP. However, HTTP carries too much unnecessary information, which takes up too much bandwidth during transmission. Even worse, fast HTTP connection needs to recreate connection channel frequently, and each connection has to go through a three-way handshake.

The RPC is the concept in a distributed system where a node calls services from nodes remotely [45]. The request detail is transparent to the user, and network connections are handled by an RPC framework automatically. This concept makes performing distributed computing tasks easier and delivers more efficient node communication. RPC frameworks generally use TCP as the transport protocol and support serialization protocols with binary encoding. In addition, since binary-encoded message is relatively small, so the data transmission takes little bandwidth.

In PolyScript, the node communication is handled by gRPC, a cross-platform and open-source RPC framework that provides high communication performance. Google initially developed this framework and used it in various enterprise-grade projects due to its exceptional performance and reliability. The serialization protocol of gRPC is protocol buffers, which is also binary encoded.

4.3.4. Data Persistence

We choose to use MongoDB as our database in this work, which is classified as a NoSQL document-oriented database. Recalling the structure of a block, it contains several attributes such as: timestamps, previous hash, and transactions. For a transaction, it has an object-like structure that contains several attributes. Moreover, the transaction data within a transaction also has an object-like structure. Traditional relational

databases store data inside tables that are not optimized for objects. Even worse, using a relational database to store the blockchain may lead to several problems, including complex relationship design, burdensome SQL commands when querying data, inflexible transaction data due to strict schema, and difficulty in horizontal scaling.

The better practice is to use a document-oriented database like MongoDB, which is optimal for storing complex objects [48]. Besides, each collection (stored object) of MongoDB is self-contained, so the records can be efficiently distributed to additional nodes for horizontal scaling.

4.3.5. Plug-and-Play Consensus Support

In a distributed system, a cluster consists of multiple nodes. To ensure data reliability, the data stored in all nodes need to be consistent. A consensus algorithm is designed to provide such consistency.

Since the selection of consensus algorithm is crucial to any blockchain system, DAPPaaS supports multiple consensus algorithms to deliver high flexibility. Specifically, we have implemented both PoW and PBFT, and the system supports to add customized consensus algorithm.

For a blockchain using PoW, nodes need to contribute computational resources to find the answer to a complex puzzle to get the right to create a new block [49]. The winning node that solves the puzzle broadcasts the new block in the P2P network, and the rest verify the authenticity by regenerating the block's hash value. After the block is verified, all nodes accept the new block and synchronize the blockchain.

PolyScript is developed in JavaScript, and solving a PoW puzzle is computationally intensive. The computation regards a dedicated JavaScript worker thread that handles the PoW puzzle to improve the system performance. Specifically, the worker thread is another instance of the JavaScript engine running in parallel.

PBFT is an efficient consensus algorithm. Suppose a P2P network contains n nodes, where f failed nodes are tolerated; if $n \geq 3f + 1$ is assured, then the rest of the nodes can reach a consensus [50]. PBFT is relatively complex to implement. Each round has a leader node, and the rest are secondary nodes. Meanwhile, there are four phases. First, "request": The leader node receives an incoming request with transactions. Second, "pre-prepare": The leader node proposes a block and broadcasts it to other secondary nodes. Third, "prepare": after nodes receive the message, they verify the message and send a "prepare" message after verifying the new block. The new block is considered "prepared" if a node receives $2f + 1$ "prepare" messages. Then, the node sends a "commit" message. Finally, "commit": if a node receives $2f + 1$ "commit" messages, the new block is accepted, and the blockchain is updated.

Users must balance the benefits and limitations choosing consensus algorithm depending on their objective and use case. PoW ensures a more secure system. Tempering a blockchain with PoW requires at least half the computational power of the entire network. Such a costly behavior prevents hackers from attacking the system. However, the computation involved with

PoW is a waste of energy; it has been criticized for being environmentally unfriendly. In contrast, PBFT is not computationally intensive. It saves energy while providing high block generation speed. However, PBFT requires at least two-thirds of the nodes in the network to be mild, degrading the system's robustness.

4.3.6. Automated Deployment

Deploying a web3 service requires setting up the running environment on a server, such as modifying system configurations and installing dependencies. Manually setting up the running environment does not meet the requirements of automated deployment. In recent years, with the advent of container technology, programs can be executed in an isolated sandbox [51]. Furthermore, the sandbox can be packaged into a container image that can be executed on another server directly without any extra configuration. Accordingly, container technology provides a function for deploying services automatically.

Docker is an open-source platform for building, managing, and deploying applications that execute inside a container [52]. In PolyScript, each component executes inside a docker container. Meanwhile, containers are launched and stopped automatically. Docker is an excellent choice as it has the following properties. First, Docker containers have strong consistency. Containers created by the same image have the same operating and file systems on any machine. Therefore, deploying the application on different platforms saves time and effort. Second, Docker containers are launched very quickly. Unlike starting up a virtual machine that takes several minutes, docker images boot up in a few seconds. Third, Docker supports automatic image generation. Docker can automatically build images from source code. Moreover, the built image supports version control, which is ideal for continuous integration and delivering tasks.

4.3.7. Automated Service Monitoring

The automated deployment can also increase the system's availability. When a service goes down, the system should identify the error immediately and replace the malfunction instance with another. We need a tool to automatically monitor and manage docker containers to achieve the goal.

Container orchestration is a technology that provides a systematic approach to control the container lifecycle. The prominent container orchestration system Kubernetes is an open-source system for automating service deployment, scaling, and management [53]. Kubernetes automatically controls and manages the usage of containers, making service deployment effortless. When an instance goes down, Kubernetes can handle the troubleshooting: the failed instance is restarted, or the related services will be transferred to another instance. In addition, Kubernetes can perform resource monitoring and scheduling. When a new instance joins the cluster, Kubernetes allocates additional resources and automatically balances incoming requests across instances.

To manage a cluster with Kubernetes, we need to set up the cluster configuration. There are two distinct approaches: the

imperative approach and the declarative approach. The imperative approach involves creating and configuring Kubernetes clusters directly from the command line tool. In comparison, the declarative approach defines cluster configuration within YAML files and then applies configurations to the cluster. In this work, we use the declarative approach because manually typing commands in the command line is not ideal. Although YAML files for cluster configuration are complex, we design a single command that can generate YAML files and start server instances automatically.

In a Kubernetes cluster, pods are the smallest units that can be deployed and managed by the system. A single pod includes one or multiple containers, which is an abstraction of a set of containers. In PolyScript, one pod is responsible for one container/service with a system dashboard to monitor the status of all running pods. There are five statuses in a pod's lifecycle: 1) "pending": the pod is configured, but containers are scheduled, 2) "running": all containers are created, and at least one container is running, 3) "succeeded": all containers are executed and terminated successfully, 4) "failed": all containers are terminated, and at least one container has terminated with an error, and 5) "unknown": the status of the pod is unknown. It may be due to a network error.

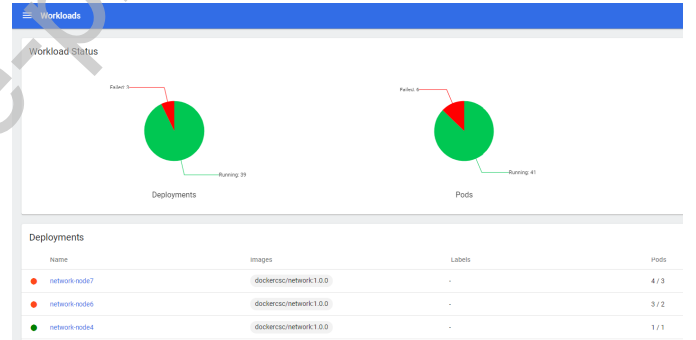


Figure 11: The cluster dashboard in the example application. It shows the component status of the whole DAPPaaS network.

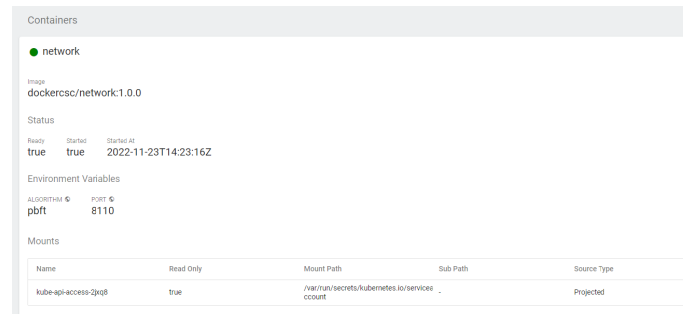


Figure 12: The container dashboard in the example application. It shows the detailed information of a specific container.

To achieve high availability, the application needs to always be running. Whenever a pod fails, Kubernetes should restarts the pod automatically. Furthermore, we can configure several pod replicas in the configuration file so that the failed pod is instantly swapped by its replica. Fig. 11 illustrates an example

of a dashboard that monitors all services. Fig. 12 is an example of detailed information of a container.

4.3.8. Other Applications: AI3 and PolyTrace

AI3: DAPPaaS-enabled Big Data Sharing. Big data sharing has become increasingly prevalent, powering applications from commercial data trading to cross-domain analytics. However, traditional platforms, such as centralized data hosting or aggregation centers, are often plagued by critical shortcomings in data privacy and authenticity. To address these challenges, we developed AI3, a DAPPaaS-based big data sharing platform in collaboration with Huawei Technologies Co., Ltd. AI3 introduces a novel dual-blockchain architecture, leveraging two loosely-coupled chains to simultaneously guarantee the privacy of raw data and the authenticity of all sharing activities.

In this ecosystem, data sharers and consumers join a network maintaining two distinct DAPPaaS instances: a metadata chain and a sharing data chain. Critically, the original data remains stored locally with the data sharer, preserving privacy. The public-facing metadata chain is used for discovery; each transaction on it records the publication timestamp, the publisher's identity, a URL pointing to the metadata file, the file type, and a hash of the original data for integrity verification. The sharing data chain, conversely, functions as an immutable audit trail. It logs every access request and fulfillment, capturing the request timestamp, the sharing timestamp, the identities of the sharer and the sharee, and a hash linking back to the corresponding entry on the metadata chain.

This dual-chain model facilitates a secure and transparent workflow. Data sharers begin by publishing descriptive metadata on the metadata chain. Prospective consumers can then search this chain using keywords to find available datasets. Once a dataset is identified, the consumer can request a data transfer, an event that is securely recorded on the sharing data chain. Furthermore, AI3 supports nearline computation, a privacy-preserving feature where consumers can execute predefined functions on a dataset with the owner's permission, without directly accessing the raw data. This computational event and its outcome are also logged on the sharing data chain, ensuring a complete and verifiable history of all data interactions.

PolyTrace: Authenticable Food Traceability. Food traceability, referring to the capacity to track a product's journey from origin to consumer, is critical for managing supply chain integrity and bolstering consumer confidence. Blockchain technology is exceptionally well-suited for this domain, as its inherent transparency and immutability provide a highly reliable foundation for verifying product provenance. Building on our collaborative work with Alibaba Group, we developed PolyTrace, a DAPPaaS-based application where stakeholders along the food supply chain form a federated network to collectively maintain the system.

This solution implements granular, role-based access control for different user types. Each transaction captures a snapshot of an item's journey, containing the submission timestamp and location, the publisher's identity, identifiers for both source and destination items, and a description for supplementary details. The system provides tailored query capabilities for

each role. Customers can perform a simple origin query to instantly verify a product's provenance. Federated members have broader access, allowing them to submit new product records as items move through the supply chain and to query detailed information about an item's full history. Administrators possess the most comprehensive access, enabling advanced end-to-end provenance tracking through sophisticated forward and backward queries that can trace an ingredient to all resulting products or deconstruct a final product back to its constituent sources.

5. Performance Evaluation

This section presents the experimental results of the DAPPaaS with a particular focus on the system throughput. We examine the performance of system through the decentralized application, PolyScript, demonstrated in Sec. 4.3. The system is implemented on two workstations for performance evaluation, each with a 64GB memory, a 1TB hard disk, and a 24-core 32-thread i9-12900 CPU, running Ubuntu 18.04 LTS operating system. The DAPPaaS employs PBFT as the consensus protocol for the nodes. We use Postman to simulate excessive and continuous API calls to the cluster. The payload of each call is a record of a student's transcript, with a data size of 1,500 bytes. Each call is regarded as a transaction with a submission time and a confirmation time on the DAPPaaS platform. In the following, we are concerned with the deployment time of additional logical nodes. The experimental results are obtained as an average of 50 repetitions. Then, we investigate how the system throughput is affected by the block generation time, block size limit, and number of nodes.

- The block generation time is an adjustable parameter defining the time difference of generating two consecutive blocks. Between two blocks' generation, the DAPPaaS network nodes need to synchronize transactions and blocks by running the consensus protocol. A long block generation time gives enough time for data synchronization and consensus; however, it may limit the system throughput. In contrast, a short block generation time may lead to an inconsistent transaction pool and even inconsistent blockchain ledger status.
- The block size limit is an adjustable parameter limiting each block's size and, thus, the number of transactions. A block that is too large may lead to network congestion. In contrast, a very small block can only contain a few transactions and may affect the system throughput.
- The number of nodes defines the network size. With a few nodes, DAPPaaS network is efficient but vulnerable to attacks. In contrast, a network that is too large incurs high communication overhead and may result in limited system throughput.

5.1. System Deployment Time

Fig. 13 shows the system deployment time with different numbers of nodes, ranging from 2 to 100. We start the sys-

tem with 2 logical nodes and record the network setup and synchronization time. When the synchronization ends, we add a logical node to connect with the existing network then measure the time for adding the node. This action is repeated until there are 100 nodes in the system. When there are 2, 50, and 100 nodes, it takes 0.13, 70.67s, and 235.42s for synchronizing the deployment, respectively.

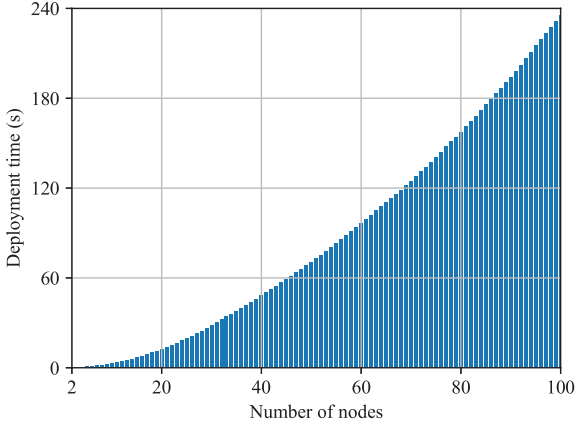


Figure 13: System deployment time vs. the number of nodes. Deploying 100 nodes demand less than 6 minutes in DAPPaaS.

It takes less than four minutes to set up a network with 100 nodes, which is efficient. Another observation is that adding a node to an existing large-scale network (e.g., 90 nodes) takes more time compared with a small-scale one (e.g., only 10 nodes). The reason is that more communication links need to be established in a larger-scale network.

5.2. Impact of Block Generation Time

In this set of experiments, we are concerned about the impact of block generation time on the system throughput. More specifically, we examine five instances of DAPPaaS networks with different block size limit (denoted with MB) and node number. We vary the block generation time from 1s to 10s with a step of 1s and evaluate the system throughput of the five DAPPaaS network instances. The result is shown in Fig. 14.

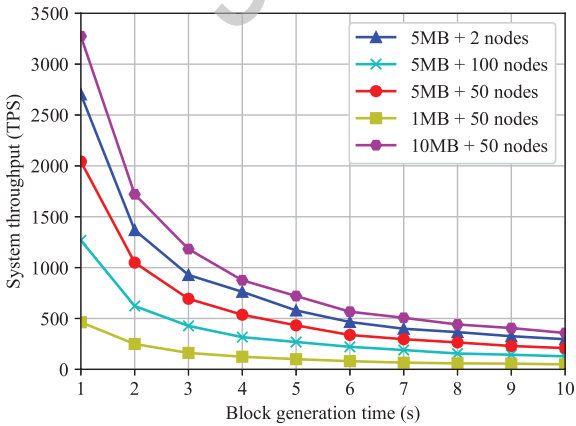


Figure 14: The impact of block generation time on system throughput in five DAPPaaS network instances.

The result indicates that the system throughput of DAPPaaS networks will decrease remarkably with a longer block generation time. The reason is that the networks have already done data synchronization and do not process any other transactions before the next round of block generation. For example, the system throughput when the block generation time is set to 1s is approximately twice compared with the one when the block generation time is 2s, despite the block size limit and number of network nodes. We assert that 1s is enough for synchronizing up to 10MB data for 100 nodes in our experimental network.

Two extreme cases in Fig. 14 are as follows. When the block generation time is only 1s, the block size limit is 10MB, and the network contains 50 nodes, the system throughput is up to 3274.6 transactions per second. The network can quickly (in 1s) synchronize excessive transactions in a large block (10MB). When the block generation time is 10s, the block size limit is 1MB, and the network of 50 nodes can only process approximately 49.0 transactions per second. The limited block size and long block generation time greatly restrict the system throughput.

5.3. Impact of Block Size Limit

The impact of the block size limit on the system throughput is investigated in this set of experiments, and the result is depicted in Fig. 15. More specifically, we examine five instances of DAPPaaS networks with different block generation time and node number settings. We vary the block size limit from 1MB to 10MB with a step of 1MB and evaluate the system throughput of the five DAPPaaS network instances.

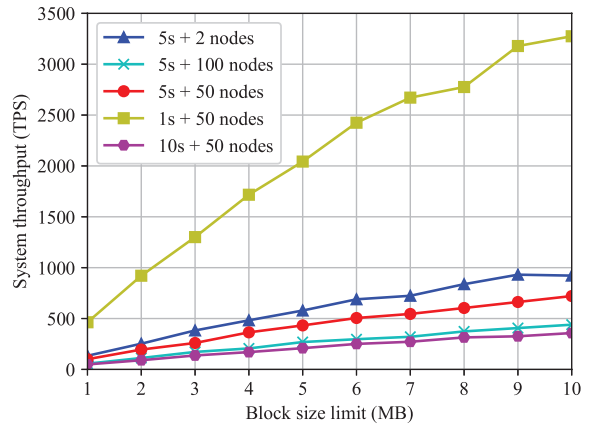


Figure 15: The impact of block size limit on system throughput in five DAPPaaS network instances.

The result indicates that the system throughput of networks will increase remarkably with a higher block size limit. The reason is that a larger block can contain more transactions. For example, the system throughput when the block size limit is set to 2MB is approximately twice of the one when the block size limit is 1MB. However, when the network contains too many nodes (e.g., 50 or 100 nodes) and each block is too large (more than 7MB), the system throughput is effected a lot because the data synchronization cost is too high, resulting in congested networks and limited performance.

5.4. Impact of Number of Nodes

Fig. 16 elaborates on the impact of the number of nodes on the system throughput. We consider five special instances of DAPPaaS networks with different block generation time and the block size limit settings. We vary the number of nodes from 2 and 20 to 100 with a step of 10 nodes and examine the system throughput of the five network instances.

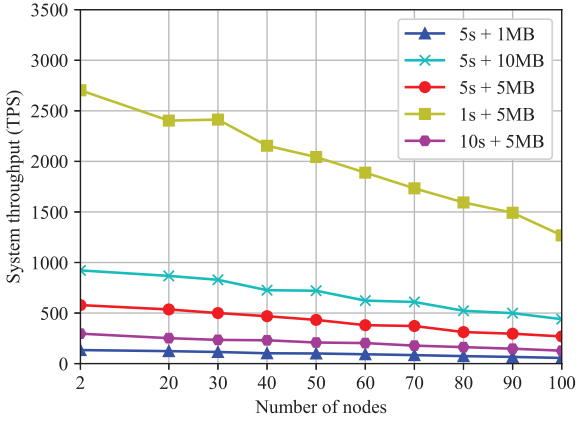


Figure 16: The impact of the number of nodes on system throughput in five DAPPaaS network instances.

The result shows that the system throughput will decrease remarkably with more network nodes. The reason is that more nodes would result in higher communication overhead and thus hinder the system throughput. However, the impact is less apparent when the network is smaller, e.g., less than 50 nodes. In contrast, the system throughput significantly drops when there are 50 nodes and more. The reason is that we are employing PBFT as the consensus protocol, and its performance is constrained by excessive nodes remarkably.

6. Conclusion and Future Directions

This work presents DAPPaaS, the first customizable blockchain as a service platform specialized for decentralized Web3 applications. We introduce the design goals and principles of DAPPaaS, following an in-depth analysis of the primary features and fundamental requirements of Web3 applications. The key technical contributions lie in the approaches enabling component modularization, communication efficiency, automated scaling and performance monitoring, distributed deployment, and resource optimization. We implemented a real-world application based on DAPPaaS and conducted extensive performance evaluations to validate its utility and efficiency. Regarding future work, we plan to conduct a more extensive performance evaluation. This will involve analyzing a broader set of metrics, including transaction confirmation latency, system availability, fault recovery time, and resource utilization rates, to provide a more holistic view of the platform's operational characteristics. Furthermore, to fully validate the customizability of the modular consensus layer, we will perform a comprehensive performance comparison with the Proof of Work consensus mechanism, offering empirical insights into

the trade-offs between different consensus strategies on the DAPPaaS platform.

7. Acknowledgments

This work was supported by the HK RGC Theme-based Research Scheme (No. T43-513/23-N) and the Research Institute for Artificial Intelligence of Things, The Hong Kong Polytechnic University.

Declaration of Interest Statement

The author is an Editorial Board Member/Editor-in-Chief/Associate Editor/Guest Editor for this journal and was not involved in the editorial review or the decision to publish this article.

References

- [1] H. Wu, H. Li, X. Luo, S. Jiang, Blockchain-based onsite activity management for smart construction process quality traceability, *IEEE Internet of Things Journal* 10 (24) (2023) 21554–21565.
- [2] F. Tschorsch, B. Scheuermann, Bitcoin and beyond: A technical survey on decentralized digital currencies, *IEEE Communications Surveys & Tutorials* 18 (3) (2016) 2084–2123.
- [3] G. Wood, et al., Ethereum: A secure decentralised generalised transaction ledger, *Ethereum Project Yellow Paper* 151 (2014) (2014) 1–32.
- [4] S. Jiang, J. Cao, H. Wu, K. Chen, X. Liu, Privacy-preserving and efficient data sharing for blockchain-based intelligent transportation systems, *Information Sciences* 635 (2023) 72–85.
- [5] M. Turkanovic, M. Holbl, K. Kosic, M. Hericko, A. Kamisalic, Eductx: A blockchain-based higher education credit platform, *IEEE Access* 6 (2018) 5112–5127.
- [6] H. Wu, S. Jiang, J. Cao, High-efficiency blockchain-based supply chain traceability, *IEEE Transactions on Intelligent Transportation Systems* 24 (4) (2023) 3748–3758.
- [7] S. Jiang, J. Cao, H. Wu, Y. Yang, M. Ma, J. He, Blochie: a blockchain-based platform for healthcare information exchange, in: *IEEE International Conference on Smart Computing (SMARTCOMP)*, 2018, pp. 49–56.
- [8] M. Conti, E. S. Kumar, C. Lal, S. Ruj, A survey on security and privacy issues of bitcoin, *IEEE Communications Surveys & Tutorials* 20 (4) (2018) 3416–3452.
- [9] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, et al., Hyperledger fabric: a distributed operating system for permissioned blockchains, in: *ACM EuroSys*, 2018, pp. 1–15.
- [10] S. Jiang, W. Chai, M. Zhang, J. Cao, S. Xuan, J. Shen, Verifying energy generation via edge llm for web3-based decentralized clean energy networks, *Information Fusion* 127 (2026) 103752.
- [11] Z. Liu, Y. Xiang, J. Shi, P. Gao, H. Wang, X. Xiao, B. Wen, Q. Li, Y.-C. Hu, Make web3. 0 connected, *IEEE Transactions on Dependable and Secure Computing* 19 (5) (2021) 2965–2981.
- [12] Y. Wang, J. Cao, S. Jiang, S. Wang, H. Liu, Z. Liang, Efficient and atomic cross-blockchain transaction processing for decentralized web3 applications, in: *IEEE Smart World Congress (SWC)*, IEEE, 2024, pp. 155–162.
- [13] S. Chaliasos, M. A. Charalambous, L. Zhou, R. Galanopoulou, A. Gervais, D. Mitropoulos, B. Livshits, Smart contract and defi security tools: Do they meet the needs of practitioners?, in: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.
- [14] F. Mlika, W. Karoui, L. B. Romdhane, Blockchain solutions for trustworthy decentralization in social networks, *Computer Networks* (2024) 110336.

- [15] R. Qin, W. Ding, J. Li, S. Guan, G. Wang, Y. Ren, Z. Qu, Web3-based decentralized autonomous organizations and operations: Architectures, models, and mechanisms, *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 53 (4) (2022) 2073–2082.
- [16] L. Cao, Decentralized ai: Edge intelligence and smart blockchain, meta-verse, web3, and desc, *IEEE Intelligent Systems* 37 (3) (2022) 6–19.
- [17] K. Gai, J. Guo, L. Zhu, S. Yu, Blockchain meets cloud computing: A survey, *IEEE Communications Surveys & Tutorials* 22 (3) (2020) 2009–2030.
- [18] C. Melo, J. Dantas, D. Oliveira, I. Fé, R. Matos, R. Dantas, R. Maciel, P. Maciel, Dependability evaluation of a blockchain-as-a-service environment, in: *IEEE Symposium on Computers and Communications (ISCC)*, IEEE, 2018, pp. 00909–00914.
- [19] M. Samaniego, U. Jamsrandorj, R. Deters, Blockchain as a service for iot, in: *IEEE iThings/GreenCom/CPSCom/SmartData*, 2016, pp. 433–436.
- [20] J. Singh, J. D. Michels, Blockchain as a service (baas): Providers and trust, in: *IEEE EuroS&P Workshops*, 2018, pp. 67–74.
- [21] A. Asheralieva, D. Niyato, Distributed dynamic resource management and pricing in the iot systems with blockchain-as-a-service and uav-enabled mobile edge computing, *IEEE Internet of Things Journal* 7 (3) (2019) 1974–1993.
- [22] G. S. Aujla, M. Singh, A. Bose, N. Kumar, G. Han, R. Buyya, Blocksdn: Blockchain-as-a-service for software defined networking in smart city applications, *IEEE Network* 34 (2) (2020) 83–91.
- [23] Y. Chen, J. Gu, S. Chen, S. Huang, X. S. Wang, A full-spectrum blockchain-as-a-service for business collaboration, in: *IEEE International Conference on Web Services*, 2019, pp. 219–223.
- [24] N. Weerasinghe, T. Hewa, M. Liyanage, S. S. Kanhere, M. Ylianttila, A novel blockchain-as-a-service (baas) platform for local 5g operators, *IEEE Open Journal of the Communications Society* 2 (2021) 575–601.
- [25] Z. Cai, G. Yang, S. Xu, C. Zang, J. Chen, P. Hang, B. Yang, Rbaas: A robust blockchain as a service paradigm in cloud-edge collaborative environment, *IEEE Access* 10 (2022) 35437–35444.
- [26] P. Bottoni, C. Di Ciccio, R. Pareschi, D. Tortola, N. Gessa, G. Massa, Blockchain-as-a-service and blockchain-as-a-partner: Implementation options for supply chain optimization, *Blockchain: Research and Applications* 4 (2) (2023) 100119.
- [27] Y. Zhao, Y. Qu, Y. Xiang, Y. Zhang, L. Gao, A lightweight model-based evolutionary consensus protocol in blockchain as a service for iot, *IEEE Transactions on Services Computing* 16 (4) (2023) 2343–2358.
- [28] Q. Lu, X. Xu, Y. Liu, I. Weber, L. Zhu, W. Zhang, ubaas: A unified blockchain as a service platform, *Future Generation Computer Systems* 101 (2019) 564–575.
- [29] W. Zheng, Z. Zheng, X. Chen, K. Dai, P. Li, R. Chen, Nutbaas: A blockchain-as-a-service platform, *IEEE Access* 7 (2019) 134422–134433.
- [30] S. Wan, H. Lin, W. Gan, J. Chen, S. Y. Philip, Web3: The next internet revolution, *IEEE Internet of Things Journal* 11 (21) (2024) 34811–34825.
- [31] S. Wang, W. Ding, J. Li, Y. Yuan, L. Ouyang, F.-Y. Wang, Decentralized autonomous organizations: Concept, model, and applications, *IEEE Transactions on Computational Social Systems* 6 (5) (2019) 870–878.
- [32] P. P. Ray, Web3: A comprehensive review on background, technologies, applications, zero-trust architectures, challenges and future directions, *Internet of Things and Cyber-Physical Systems* 3 (2023) 213–248.
- [33] L. Stockburger, G. Kokosioulis, A. Mulkamala, R. R. Mulkamala, M. Avital, Blockchain-enabled decentralized identity management: The case of self-sovereign identity in public transportation, *Blockchain: Research and Applications* 2 (2) (2021) 100014.
- [34] S. Houy, P. Schmid, A. Bartel, Security aspects of cryptocurrency wallets—a systematic literature review, *ACM Computing Surveys* 56 (1) (2023) 1–31.
- [35] T. Berners-Lee, R. Cailliau, A. Luotonen, H. F. Nielsen, A. Secret, The world-wide web, *Communications of the ACM* 37 (8) (1994) 76–82.
- [36] S. Aghaei, M. A. Nematbakhsh, H. K. Farsani, Evolution of the world wide web: From web 1.0 to web 4.0, *International Journal of Web & Semantic Technology* 3 (1) (2012) 1–10.
- [37] N. Shadbolt, T. Berners-Lee, W. Hall, The semantic web revisited, *IEEE Intelligent Systems* 21 (3) (2006) 96–101.
- [38] T. Berners-Lee, J. Hendler, O. Lassila, The semantic web: A new form of web content that is meaningful to computers will unleash a revolution of new possibilities, in: *Linking the World’s Information: Essays on Tim Berners-Lee’s Invention of the World Wide Web*, ACM, 2023, pp. 91–103.
- [39] W. Gan, Z. Ye, S. Wan, P. S. Yu, Web 3.0: The future of internet, in: *Companion Proceedings of the ACM Web Conference 2023*, 2023, pp. 1266–1275.
- [40] Y. Lin, Z. Gao, Y. Tu, H. Du, D. Niyato, J. Kang, H. Yang, A blockchain-based semantic exchange framework for web 3.0 toward participatory economy, *IEEE Communications Magazine* 61 (8) (2023) 94–100.
- [41] M. Zhang, J. Cao, Y. Sahni, Q. Chen, S. Jiang, L. Yang, Blockchain-based collaborative edge intelligence for trustworthy and real-time video surveillance, *IEEE Transactions on Industrial Informatics* 19 (2) (2022) 1623–1633.
- [42] Y. Liu, J. Wang, Z. Yan, Z. Wan, R. Jäntti, A survey on blockchain-based trust management for internet of things, *IEEE Internet of Things Journal* 10 (7) (2023) 5898–5922.
- [43] S. Jiang, J. Cao, H. Wu, Fast and atomic cross-blockchain asset exchange for metaverse interoperability, in: *2023 IEEE International Conference on Metaverse Computing, Networking and Applications (MetaCom)*, IEEE, 2023, pp. 177–184.
- [44] S. Jiang, J. Cao, C. L. Tung, Y. Wang, S. Wang, Sharon: Secure and efficient cross-shard transaction processing via shard rotation, in: *2024 IEEE International Conference on Computer Communications (INFOCOM)*, IEEE, 2024, pp. 2418–2427.
- [45] A. D. Birrell, B. J. Nelson, Implementing remote procedure calls, *ACM Transactions on Computer Systems* 2 (1) (1984) 39–59.
- [46] S. Jiang, J. Cao, H. Wu, Y. Yang, Fairness-based packing of industrial iot data in permissioned blockchains, *IEEE Transactions on Industrial Informatics* 17 (11) (2020) 7639–7649.
- [47] S. Pallevatta, V. Kostakos, R. Buyya, Placement of microservices-based iot applications in fog computing: A taxonomy and future directions, *ACM Computing Surveys* 55 (14s) (2023) 1–43.
- [48] Z. Gui, K. G. Paterson, T. Tang, Security analysis of {MongoDB} queryable encryption, in: *32nd USENIX Security Symposium (USENIX Security)*, 2023, pp. 7445–7462.
- [49] J. Xu, C. Wang, X. Jia, A survey of blockchain consensus protocols, *ACM Computing Surveys* 55 (13s) (2023) 1–35.
- [50] M. Castro, B. Liskov, Practical byzantine fault tolerance and proactive recovery, *ACM Transactions on Computer Systems* 20 (4) (2002) 398–461.
- [51] R. Malhotra, A. Bansal, M. Kessentini, A systematic literature review on maintenance of software containers, *ACM Computing Surveys* 56 (8) (2024) 193:1–193:38.
- [52] N. Singh, Y. Hamid, S. Juneja, G. Srivastava, G. Dhiman, T. R. Gadekallu, M. A. Shah, Load balancing and service discovery using docker swarm for microservice based big data applications, *Journal of Cloud Computing* 12 (1) (2023) 4.
- [53] C. Carrión, Kubernetes scheduling: Taxonomy, ongoing issues and challenges, *ACM Computing Surveys* 55 (7) (2022) 1–37.